



Curso Profissional de Técnico de Gestão e Programação de
Sistemas Informáticos

Relatório da

Prova de Aptidão Profissional

Ano Letivo de 2025/2026

Gestão de Práticas Religiosas e do Quotidiano
(React, Node.JS, MYSQL)

O formando: Telmo José da Silva Gonçalves

Ano letivo 2025/2026

Índice

Índice	1
Percurso e Interesse Pessoal	2
Ponto de Partida	2
Relevância do Tema	2
Destinatários Alvos	2
Finalidades do Projeto	3
Fontes de Informação e Escolha das Ferramentas	3
Resumo	4
Introdução	5
Desenvolvimento	8
Organização do Projeto e Separação	8
Arquitetura cliente-servidor e Comunicação	9
Base de dados: Normalização e Integridade	10
Autenticação e Segurança	10
Gamificação e Integração de C	11
Interface e Experiência de utilizador	11
ADICIONAR IMAGENS DO ANDROID - PELO MENOS 2 OU 3	13
Controlo de Versões	13
Ferramentas e Recursos Utilizados	13
Conclusão	14
Glossário/Vocabulário	15
Bibliografia	16

Resumo

O Lifinity é uma aplicação web e móvel de produtividade pessoal com ludificação, comunidade e inspiração diária que permite aos utilizadores definirem objetivos com práticas associadas, assim como motivá-los através de comunidades e estatísticas de evolução. Nesta aplicação, é possível participar em comunidades, criar e realizar atividades, visualizar gráficos comparáveis entre comunidades, gerir perfil, partilhar inspirações religiosos, recorrer a um chatbot para ajuda e um ranking para competição saudável.

O projeto foi desenvolvido como uma aplicação full stack, dividida em:

- Frontend (Cliente): interface, onde ficam as páginas, componentes, imagens e configuração visual (desenvolvida em React e Recharts para os gráficos, assim como Tailwind CSS), em produção real precisaria de um domínio DNS configurado e DHCP na rede de alojamento;
- Aplicação Android (Cliente móvel): versão nativa da aplicação para Android, desenvolvida em Java com interfaces em XML, que comunica com o mesmo servidor através da biblioteca Retrofit, permite usar o Lifinity também no telemóvel;
- Backend (Servidor): gestão e manutenção de atividades (desenvolvido em Node.js e com a framework Express.js), contém as rotas, controllers, configuração da base de dados e middlewares de autenticação baseados em JWT (JSON Web Tokens). As palavras-passe são protegidas com bcrypt e a base de dados é acedida com prepared statements para prevenir injeção de SQL. É um servidor acessível por nome/endereço, e usa XAMPP local, logo implica resolução DNS e resolução de nomes local;
- Base de dados: Estruturada para garantir integridade e evitar dados soltos (MySQL) ;
- Módulo Nativo em C: cálculos ligados à ludificação e estatísticas (integração feita através de N-API e node-gyp), integração da primeira linguagem que já estudei;
- Git e Github: usado para controle de versões e guardar o projeto remotamente.

Percurso e Interesse Pessoal

Ponto de Partida

O tema que escolhi para a minha prova de aptidão profissional foi o desenvolvimento de uma aplicação de produtividade pessoal com componentes de ludificação[01], comunidade e inspiração diária - Lifinity[00]. Resumidamente, trata-se de uma aplicação multiplataforma (web e Android) que permite ao utilizador gerir tarefas e objetivos, acompanhar a sua evolução através de estatísticas e níveis, e manter-se motivado através de elementos de jogo e de interação com outros utilizadores.

Relevância do Tema

O motivo pelo qual escolhi este tema parte de, em primeiro lugar, um interesse pessoal pela organização e pela produtividade pessoal, mas vai além disso. Hoje em dia, grande parte dos adolescentes e jovens está habituada à ludificação em quase tudo o que utiliza e/ou faz, desde redes sociais a aplicações de estudo ou aprendizagens. Essa habituação acaba por criar uma certa dependência de recompensas imediatas, a pessoa tem dificuldade em realizar tarefas que não tragam um benefício concreto e visível a curto prazo. No fundo, traduz-se numa falta de disciplina e de motivação.

Reconheço este problema porque, obviamente, também já me encontrei nessa situação várias vezes e foi precisamente daí que surgiu a ideia: em vez de lutar contra o hábito (ainda deve ser feito, não o devemos negligenciar), porque não usá-lo a favor do utilizador?

Destinatários Alvos

O Lifinity destina-se a qualquer pessoa que queira organizar as suas tarefas e objetivos de uma maneira mais motivadora. Mas tem como público preferencial os adolescentes e jovens adultos, mas não que qualquer pessoa fora desse “espectro”[02] não seja bem vinda a utilizar a aplicação.

Finalidades do Projeto

As finalidades que me propus atingir com este projeto foram:

- Desenvolver uma aplicação Full Stack[03] Funcional, que integrasse frontend[04], backend[05] e base de dados[06];
- Aplicar, no projeto real, os conhecimentos adquiridos ao longo dos três anos do curso (programação[07], redes e bases de dados);
- Criar uma aplicação multiplataforma, disponível tanto em navegador web como em Android;
- Implementar boas práticas de organização de código e de segurança;
- Tornar a gestão de tarefas mais motivadora através da ludificação e de uma componente social, que ajude o utilizador a desenvolver disciplina[08].

Fontes de Informação e Escolha das Ferramentas

A razão para eu ter escolhido as ferramentas que escolhi para o desenvolvimento deste projeto, têm como base principalmente o facto de que a OCRAMClima[09], empresa cuja eu estagiei no terceiro ano de curso, durante pouco mais de um mês, tinham uma componente de programação, que eram especializados em Node.js[10], Express[11], React[12] e Tailwind CSS[13]. Ajudaram-me bastante nos fundamentos do projeto então decidi continuar a utilizar essas ferramentas, até porque javascript[14] é uma linguagem que aprendemos/utilizamos neste terceiro ano de curso, o que ajudaria na compreensão e estruturação dos códigos. O mesmo pode ser dito para CSS. Decidi também utilizar MySQL[15] como base de dados relacional, visto que aprendemos também na disciplina de programação, e finalmente, o módulo em C[16] foi incluído para integrar a primeira linguagem aprendida no curso.

Introdução

O projeto foi desenvolvido no âmbito da Prova de Aptidão Profissional, e envolve conceitos abordados ao longo dos três anos do curso de TGPSP [\[17\]](#), nomeadamente das disciplinas de Programação de Sistemas Informáticos, Redes de comunicação e Sistemas Operativos.

Antes de passar ao desenvolvimento em concreto da aplicação web, limitei-me a fazer planeamentos para cada “fase” do projeto, desde a definição inicial dos requisitos até a preparação da defesa. Assim como um diagrama de entidade-relacionamento [\[18\]](#) para a base de dados relacional que por sinal veio a demonstrar-se muito útil para a criação do SQL [\[19\]](#).

#	Fase / Tarefa	Início	Fim	Duração
1	Planeamento e definição de requisitos	01/10/2025	15/10/2025	2 sem
2	Desenho da base de dados (ER + tabelas)	16/10/2025	31/10/2025	2 sem
3	Backend — autenticação (JWT, bcrypt)	01/11/2025	20/11/2025	3 sem
4	Backend — API de tarefas (CRUD)	21/11/2025	10/12/2025	3 sem
5	Frontend web — estrutura e autenticação	11/12/2025	31/12/2025	3 sem
6	Frontend web — gestão de tarefas	01/01/2026	20/01/2026	3 sem
7	Ludificação (XP, níveis, ranking) + módulo C	21/01/2026	10/02/2026	3 sem
8	Comunidade (grupos, amigos, chat)	11/02/2026	05/03/2026	3,5 sem
9	Inspiração diária + Assistente IA	06/03/2026	20/03/2026	2 sem
10	Estatísticas e gráficos	21/03/2026	05/04/2026	2 sem
11	Aplicação Android — ecrãs principais	06/04/2026	30/04/2026	3,5 sem
12	Aplicação Android — comunidade e refinamento	01/05/2026	20/05/2026	3 sem
13	Moderação, filtros e melhorias finais	21/05/2026	05/06/2026	2 sem
14	Relatório e Preparação	06/06/2026	20/06/2026	2 sem
15	Testes, correção de bugs e documentação	21/06/2026	30/06/2026	1,5 sem

Tabela 1 — Cronograma de desenvolvimento do projeto (Gantt)

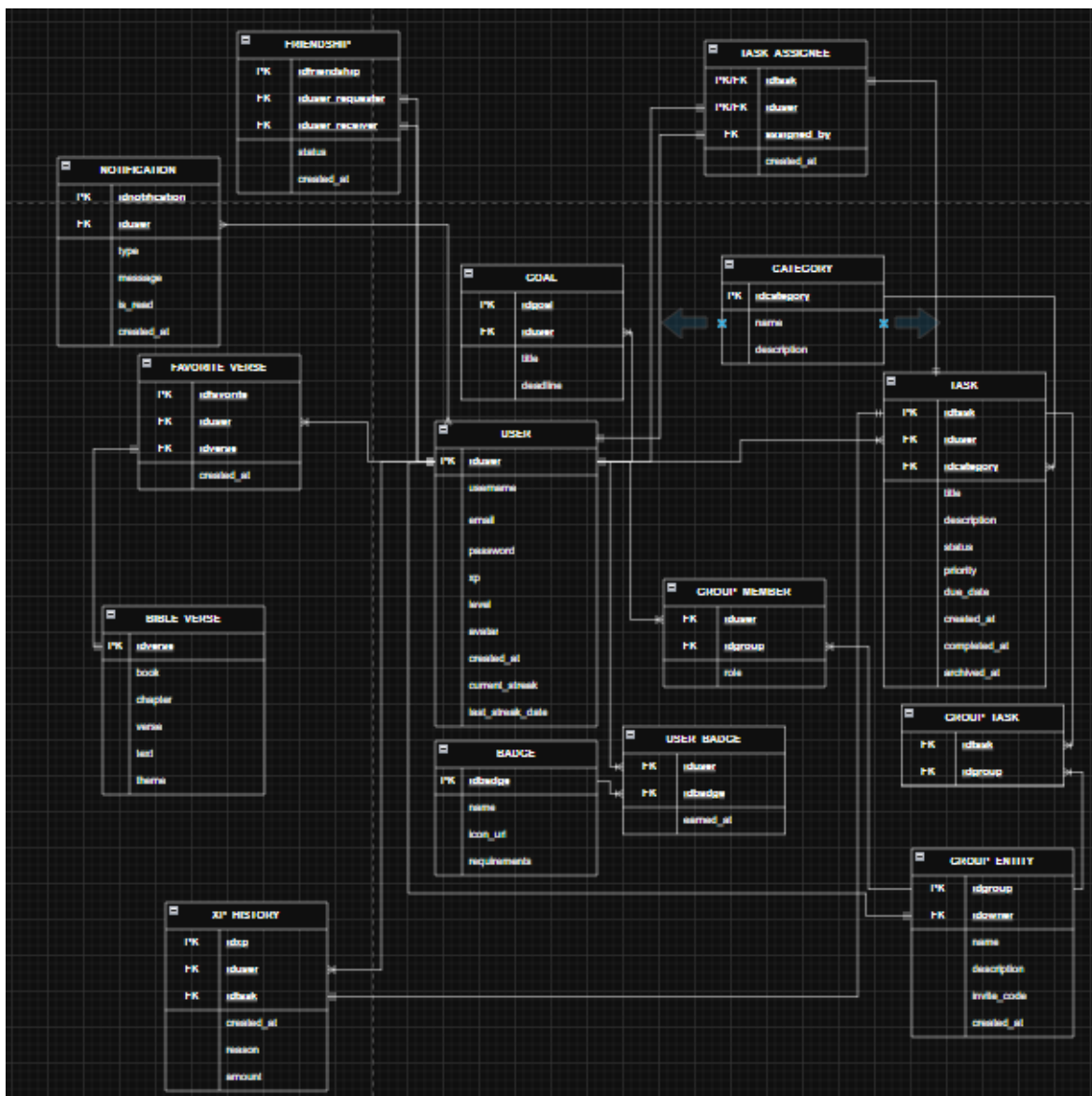


Figura 1 — Diagrama de Tabelas do modelo Relacional da base de dados Lifinity

É importante referir que este diagrama de tabelas[20] não é o final, mas sim o primeiro de todos que planeei, logo é de esperar muitas diferenças em nomes e tamanho em comparação com o diagrama que é possível encontrar dentro das pastas do projeto.

Além disso, mesmo antes de começar a desenvolver o projeto, foi definida a estrutura de navegação da aplicação, ou seja, a forma como as diferentes páginas e menus se ligam entre si. O objetivo era que a aplicação funcionasse de forma intuitiva, de maneira que levasse o utilizador desde a entrada até às funcionalidades principais sem confusão:

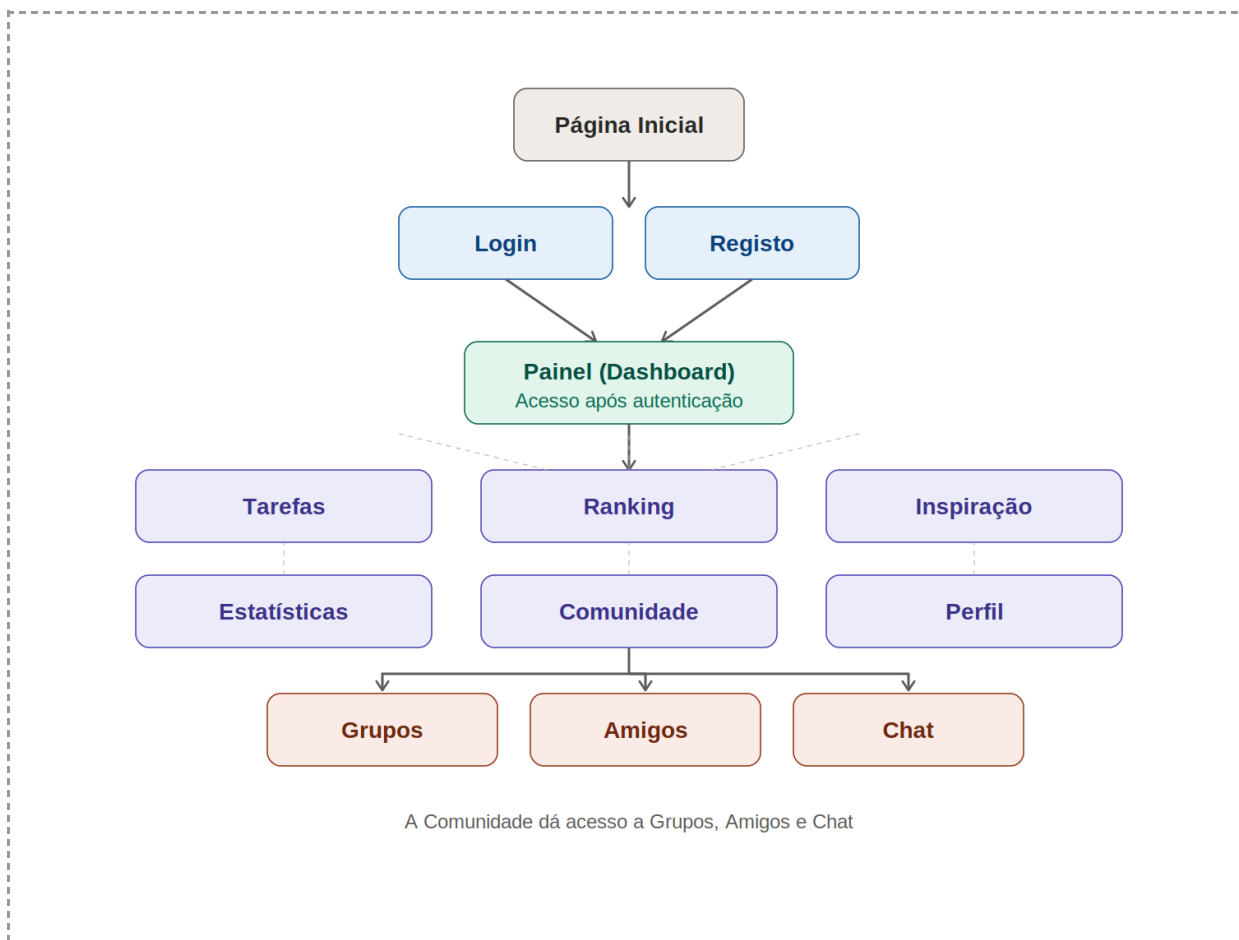


Figura 2 — Esquema de navegação / estrutura da aplicação

O fluxo previsto é o seguinte: o utilizador chega à página inicial, onde pode optar por iniciar sessão ou registar-se. Após a autenticação, é encaminhado para o painel principal (dashboard), que serve de ponto central de acesso a todas as funcionalidades. A partir deste painel, o utilizador alcança os seis menus principais: Tarefas (a página de entrada por defeito), Ranking, Inspiração, Estatísticas, Comunidade e Perfil. A Comunidade funciona como um subnível que dá acesso às funcionalidades sociais da aplicação: Grupos, Amigos e Chat.

O logótipo do Lifinity é representado por um Ouroboros^[21], isto é, a figura de uma serpente que morde a própria cauda e forma um ciclo contínuo. O Ouroboros simboliza o ciclo, a continuidade e a renovação infinita, conceitos que se ligam diretamente à minha ideia central da aplicação:

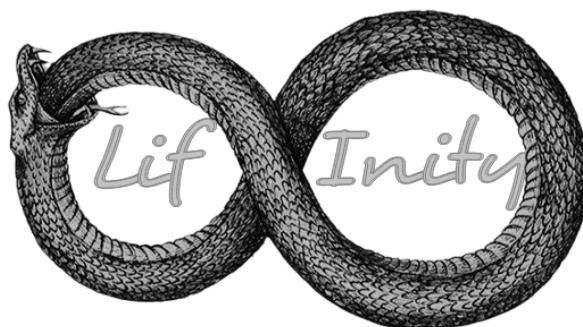


Figura 3 — Logótipo do Lifinity

A própria forma da serpente, ao curvar-se, evoca o símbolo do infinito, reforça o nome "Lifinity", que resulta da junção das palavras "life" (vida) e "infinity" (infinito).

A sugestão de utilizar esta figura partiu de um amigo, e a ideia acabou por fazer sentido. A imagem base da serpente foi obtida a partir de uma pesquisa online e, logo após, editada no Photopea, uma ferramenta de edição de imagem gratuita e semelhante ao Photoshop. Onde adicionei o nome da aplicação na própria forma do logótipo, dividido entre as duas curvas da serpente.

Desenvolvimento

No desenvolvimento do Lifinity, tentei seguir as fases definidas no planeamento, assim como um conjunto de boas práticas de engenharia de software[22], para produzir uma aplicação organizada, segura e preparada para crescer.

Organização do Projeto e Separação

A primeira decisão foi organizar o projeto de forma clara, separei-o em quatro partes: O backend (servidor e API[23]), o frontend (interface), a aplicação android e a pasta de documentação.

Dentro do backend, o código foi dividido por “responsabilidades”, as rotas[24] definem os endpoints[25], os controllers[26] contêm a lógica de cada funcionalidade, os

middlewares[27] tratam da autenticação e a configuração guarda a ligação à base de dados. Esta separação serviu para evitar que eu concentrasse tudo num único ficheiro, para tornar o código mais legível e mais fácil de gerir.

No frontend apliquei a mesma “filosofia”: as páginas ficaram separadas dos componentes reutilizáveis. Sempre que um elemento visual era usado várias vezes, como botões, cartões ou modais, foi transformado em componente reutilizável, para evitar a redundância[28] de código e garantir que uma alteração num único sítio reflete em toda a aplicação. Ou seja, tentei não repetir lógica nem estrutura desnecessariamente.

Já a aplicação Android, ficou na pasta própria e seguiu a organização típica de um projeto Android nativo (ou pelo menos como o professor nos ensinou), com as Activities, os Adapters das listas e os layouts em XML[29] separados. Apesar de ser uma plataforma diferente, seguiu o mesmo princípio de separação de responsabilidades e comunicou com o mesmo backend, para aproveitar toda a API que já tinha criado.

Arquitetura cliente-servidor e Comunicação

O Lifinity é composto por uma arquitetura cliente-servidor[30], assim como já mencionei antes. O cliente nunca acede diretamente à base de dados, mas sim através de pedidos HTTP[31], e o servidor responde em formato JSON[32], igual a uma API REST[33]. Essa separação deu-me a vantagem do mesmo backend servir para tanto a aplicação web como a aplicação android. No frontend, os pedidos foram feitos com a biblioteca Axios[34]; na aplicação android, com a biblioteca Retrofit[35], mas ambos comunicam com a mesma API.

Em ambiente de desenvolvimento, esta comunicação dependia da resolução de nomes: o servidor corre em localhost[36] (e foi traduzido localmente para 127.0.0.1 pelo ficheiro hosts[37]) e a aplicação Android, no emulador, acedia ao servidor pelo endereço 10.0.2.2 . Se eu implantar o projeto realmente, isto teria de ser feito através de um servidor DNS e um domínio próprio.

Base de dados: Normalização e Integridade

Eu desenhei a base de dados em MySQL e segui os princípios de normalização[38], principalmente até à 3ª Forma Normal[39], da maneira que nos foi ensinada pela professora. O objetivo era evitar a repetição desnecessária de dados, separar as entidades por responsabilidade e manter a integridade da informação. Um pequeno exemplo é que os versículos são guardados uma única vez e ligados aos utilizadores através de uma tabela de favoritos, para evitar duplicar o texto.

A integridade referencial[40] é garantida através de chaves primárias[41] e estrangeiras[42]. Para as relações de muitos-para-muitos[43] criei tabelas intermédias e usei regras como ON DELETE CASCADE[44] e ON DELETE SET NULL[45], conforme a necessidade de usá-las.

Houve também decisões de desenho específicas para evitar problemas, por exemplo a tabela dos grupos, que foi chamada GROUP_ENTITY porque GROUP é uma palavra reservada em SQL, e fiquei com receio de dar conflito. Além disso, também fiz uma distinção entre apagar e ocultar uma tarefa, se ela tiver sido concluída, não é eliminada da base de dados, apenas ocultada da vista do utilizador, para que não se percam dados e para preservar o histórico para as estatísticas e enfim...

Autenticação e Segurança

Tentei tratar da segurança da melhor maneira possível, até com a ajuda do professor. A autenticação foi implementada com JWT[46] (JSON Web Tokens). Após o login, o servidor gera um token que o cliente guarda e envia em cada pedido, depois um middleware valida esse token antes de dar-lhe acesso a rotas protegidas, para garantir que cada utilizador só pode aceder aos seus próprios dados.

As palavras-passe nunca são guardadas em texto simples. São protegidas com bcrypt, que aplica uma função de hash[47] com salt[48] antes de armazená-las. Utilizei também prepared statements[49], onde os valores introduzidos pelo utilizador são tratados como dados e nunca como parte do comando SQL, para prevenir injeções de SQL[50]. Foi feita também a validação dos dados recebidos do cliente antes de serem processados.

Gamificação e Integração de C

Para diferenciar o projeto, eu fiz parte dos cálculos da ludificação (a atribuição de XP e o cálculo de níveis) num módulo escrito em C, ligado ao backend através de N-API e compilado com node-gyp[51]. Honestamente só decidi fazê-lo para mostrar que pelo menos lembro-me do que aprendi da primeira linguagem estudada no curso, C.

Interface e Experiência de utilizador

Como já referido no resumo, o frontend foi desenvolvido em React com Vite[52], o que ajudou um pouco no desenvolvimento, por conta da atualização instantânea da interface, bastava guardar e dar refresh e estava atualizado. A parte da estilização foi feita com Tailwind CSS. A navegação entre páginas usa o React Router, onde as páginas privadas ficam dentro de um layout principal que verifica a autenticação e mantém o menu sempre visível. Construí os gráficos de estatísticas com a biblioteca Recharts. A nível visual, optei por uma estética mais escura, já que foi a mesma estética que escolhi para a apresentação do powerpoint (além da maioria das aplicações hoje em dia serem mais escuras visualmente).

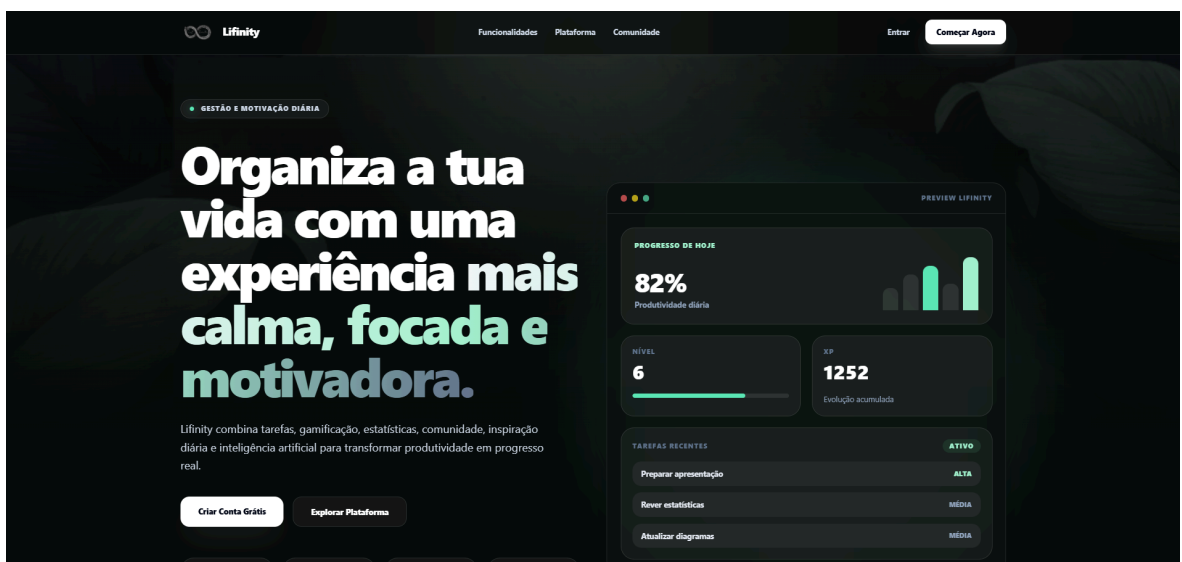


Figura 4 — LandingPage[53] do Lifinity

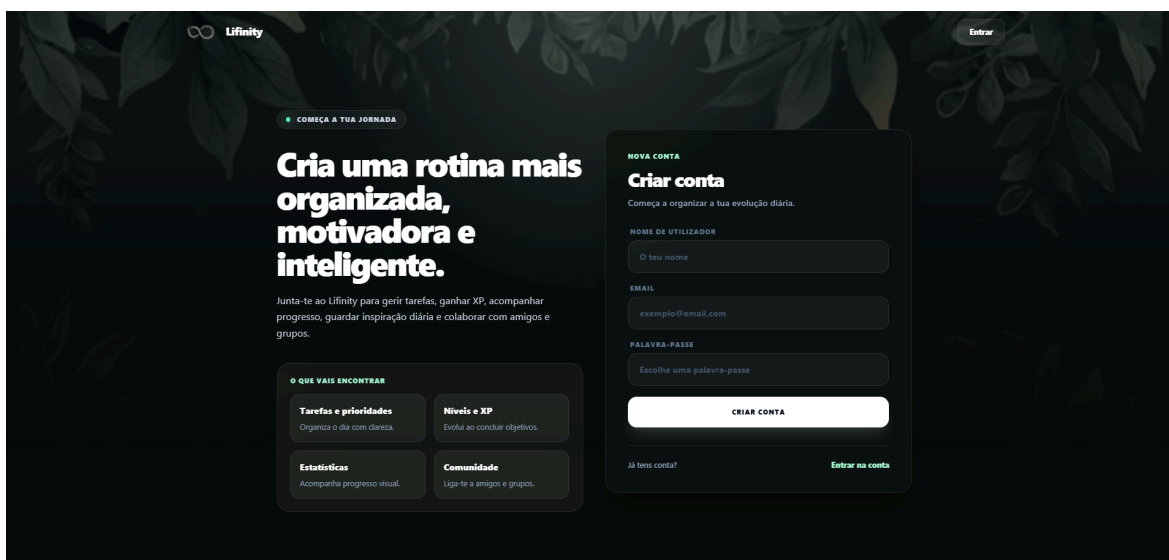


Figura 5 — Página de Registo do Lifinity



Figura 6 — Página de Inspiração do Lifinity

ADICIONAR IMAGENS DO ANDROID - PELO MENOS 2 OU 3

Controlo de Versões

Durante todo o desenvolvimento, utilizei o [Git\[54\]](#) para criar várias versões distintas do projeto, e fiz commits para cada nova funcionalidade, que era integrada no ramo principal apenas após verificar que estava tudo correto. Isto permitiu-me trabalhar em funcionalidades diferentes AO MESMO TEMPO, sem preocupar-me com a versão que funcionava, é realmente algo necessário para se trabalhar, principalmente em equipa.

Ferramentas e Recursos Utilizados

Novamente, no desenvolvimento foram utilizadas as seguintes ferramentas: Visual Studio Code[\[55\]](#) como editor de código, o XAMPP para executar o MySQL localmente e o phpMyAdmin para gerir a base de dados visualmente, o Node.js como ambiente de execução do servidor, e o Git/Github para controle de versões.

Conclusão

Com o desenvolvimento do Lifinity, consegui atingir os objetivos a que me propus: criar uma aplicação full stack funcional e multiplataforma, que junta numa só ferramenta a gestão de tarefas, a ludificação, as estatísticas, a inspiração diária e uma componente social.

Ao longo do projeto, aprendi honestamente muito mais do que esperava, uma vez que melhorei os conhecimentos das várias disciplinas do curso e percebi como elas se ligam na prática. A programação, as redes e as bases de dados deixaram de ser temas separados para passarem a fazer parte de um todo. Aprendi também a organizar um projeto grande, a separar responsabilidades no código, a evitar repetição e a manter tudo em versões com o Git, o que me deu uma noção real de como se trabalha num projeto de software a sério.

As maiores dificuldades surgiram, sobretudo, fora do que tinha sido ensinado diretamente, a configuração e compilação do módulo em C com o node-gyp, alguns problemas de permissões

no sistema, e garantir que a comunicação entre a aplicação Android e o backend funcionava em ambiente de desenvolvimento. A distinção entre apagar e ocultar tarefas, para não perder dados das estatísticas, foi outro ponto que me obrigou a pensar melhor na estrutura da base de dados. Cada uma destas dificuldades, no entanto, acabou por melhorar o projeto, porque me obrigou a corrigir e a repensar decisões.

Quanto à implementação real, o Lifinity está totalmente funcional no ambiente local. Para ser colocado verdadeiramente em produção, como já mencionei antes, seria necessário alojá-lo num servidor com um domínio próprio e registos DNS configurados, e reforçar a comunicação com HTTPS, para que qualquer pessoa pudesse aceder à aplicação a partir da Internet, em vez de apenas localmente.

Em relação a melhorias futuras, há algumas coisas que tenho em mente, além da produção em concreto, por exemplo completar e melhorar a aplicação Android, implementar a conclusão individual de atividades em grupo (penso na possibilidade de, ao criar uma atividade direcionada a um grupo, ser possível cada utilizador concluí-la individualmente, ao invés da atual conclusão singular, onde se qualquer utilizador completa a atividade, ela fecha), e aprofundar a comparação de estatísticas entre amigos e grupos. De uma forma geral, considero que o projeto cumpriu o seu propósito, não só como prova final do curso, mas também como uma ferramenta em que acredito e que eu próprio gostaria de usar.

Glossário/Vocabulário

[00] Lifinity - O nome que decidi dar ao meu projeto, literalmente a junção de Life (vida, existência,...) e Infinity (infinito, infinidade,...).

[01] Ludificação - Supostamente a maneira correta de dizer gamificação em português de Portugal, significa a aplicação de mecânicas, dinâmicas e características inspiradas em jogos, em contextos do mundo real.

[02] Espectro - Utilizei “espectro” no contexto de uma faixa etária ou ao conjunto de pessoas abrangidas por alguma coisa (neste caso o público alvo)

[03] Full Stack - É basicamente a capacidade de desenvolver uma aplicação de software completa que abrange tanto a interface com o utilizador quanto a lógica e as bases de dados do projeto.

[04] Frontend - A parte visual e interativa de um site, sistema ou aplicação, resumidamente, é onde o utilizador interage.

[05] Backend - O que está oculto, num sistema ou aplicação. Gosto de pensar como o motor que processa os dados e gere a lógica.

[06] Base de Dados - Literalmente um conjunto organizado de dados armazenados para facilitar a gestão e acesso aos mesmos dados.

[07] Programação - A definição é o processo de criar um conjunto de instruções (scripts) estruturadas que dizem ao computador como fazer algumas tarefas específicas (A base de todos os pontos anteriores é a programação).

[08] Disciplina - A disciplina é a capacidade de cumprir regras, mantendo o foco total para atingir um objetivo, mesmo sem vontade, motivação e mesmo que requeira sacrifícios.

[09] OCRAMClima - É a empresa onde realizei o estágio do terceiro ano de curso, durante pouco mais de um mês, e que tinha uma componente de programação especializada nas tecnologias que vim a usar neste projeto.

[10] Node.js - Ambiente de execução que permite correr código JavaScript fora do navegador cujo utilizei e também era utilizado pelos trabalhadores da OCRAMClima.

[11] Express - Um framework para Node.js que simplifica a criação de servidores web e de APIs, facilita bastante o uso de Node.js (também aconselhado pelos trabalhadores do estágio).

[12] React - Uma biblioteca de JavaScript usada para construir interfaces de utilizador, baseada em componentes reutilizáveis, basicamente permite a atualização instantânea das páginas sem ser preciso fazer nada além de salvar.

[13] Tailwind CSS - Um framework de CSS baseado em classes utilitárias, resumidamente permite estilizar os elementos diretamente no mesmo ficheiro, sem ter que escrever tanto em um ficheiro separado.

[14] JavaScript - A linguagem que acabou por ser a “espinha dorsal” do projeto, usei-a tanto no frontend como no backend. É a linguagem da web por excelência e, como já a tínhamos visto no curso, senti-me à vontade para a usar em todo o lado.

[15] MySQL - O sistema de base de dados relacional que escolhi. Organiza tudo em tabelas que se relacionam entre si e, honestamente, foi a escolha óbvia por ter sido o que aprendemos na disciplina de programação.

[16] Módulo em C - Um pequeno pedaço do projeto que escrevi em C e liguei ao backend, responsável pelos cálculos da ludificação. C é uma linguagem mais “baixa” e próxima da máquina, e foi a primeira que aprendi no curso, daí ter feito questão de a incluir.

[17] TGPSI - A sigla do meu curso, “Técnico de Gestão e Programação de Sistemas Informáticos”.

[18] Diagrama de entidade-relacionamento (ER) - Um desenho que mostra as entidades da base de dados (como utilizadores ou tarefas) e como se relacionam entre si. Usei-o para planear a base de dados antes sequer de a criar, e poupou-me muitas dores de cabeça.

[19] SQL - A sigla de “Structured Query Language”, basicamente a linguagem que se usa para falar com a base de dados, seja para criar, consultar ou alterar dados.

[20] Diagrama de tabelas - Parecido com o diagrama ER, mas já mais “real”: mostra as tabelas tal como ficam na base de dados, com as suas colunas e ligações. É um passo mais próximo da implementação.

[21] Ouroboros - Aquele símbolo antigo da serpente que morde a própria cauda e forma um círculo. Representa o ciclo eterno e a renovação infinita, e foi precisamente por isso que o escolhi para o logótipo.

[22] Engenharia de software - A área que trata de desenvolver software seguindo métodos e boas práticas, para que ele fique organizado, fiável e fácil de manter, em vez de ser só “código que funciona”.

[23] API - Sigla de “Application Programming Interface”. Gosto de pensar nela como a ponte que permite que duas aplicações falem uma com a outra; no meu caso, entre o cliente e o servidor.

[24] Rotas - Os vários “caminhos” que o servidor disponibiliza, cada um ligado a uma funcionalidade. Por exemplo, há uma rota para criar tarefas e outra para iniciar sessão.

[25] Endpoints - Os pontos de acesso de uma API, ou seja, cada endereço específico ao qual o cliente faz um pedido para obter ou enviar dados.

[26] Controllers - Os ficheiros do backend onde mora a lógica de cada funcionalidade. São eles que recebem os pedidos das rotas, fazem o trabalho (como consultar a base de dados) e devolvem a resposta.

[27] Middlewares - Funções que “ficam no meio do caminho” de um pedido antes de ele chegar ao destino. Usei-os sobretudo para verificar se o utilizador está autenticado e proteger as rotas.

[28] Redundância - Repetição desnecessária de código ou de informação. Tentei evitá-la ao máximo, porque assim, quando preciso de mudar algo, só tenho de o fazer num sítio.

[29] XML - Sigla de “eXtensible Markup Language”. Na aplicação Android, foi a linguagem que usei para definir como ficam os ecrãs, ou seja, os layouts.

[30] Arquitetura cliente-servidor - O modelo em que um lado (o cliente) faz pedidos e o outro (o servidor) responde. O cliente nunca toca diretamente nos dados, vai sempre buscá-los através do servidor.

[31] HTTP - Sigla de “HyperText Transfer Protocol”, o protocolo que a web usa para o cliente e o servidor trocarem pedidos e respostas.

[32] JSON - Sigla de “JavaScript Object Notation”. É um formato de texto simples para trocar dados entre cliente e servidor, fácil de ler tanto para mim como para a máquina.

[33] API REST - Um estilo de organizar uma API em que cada funcionalidade tem o seu endpoint e os dados viajam normalmente em JSON, seguindo regras padronizadas sobre HTTP. Foi o estilo que segui.

[34] Axios - A biblioteca que usei no frontend para fazer os pedidos ao servidor de forma simples, sem complicar.

[35] Retrofit - O equivalente ao Axios, mas do lado do Android (em Java). Serve para a aplicação móvel comunicar com a mesma API.

[36] localhost - O nome que se refere ao próprio computador onde a aplicação está a correr. É traduzido para o endereço 127.0.0.1.

[37] Ficheiro hosts - Um ficheiro do sistema que associa nomes a endereços IP localmente, permitindo resolver nomes sem precisar de um servidor DNS externo.

Telmo Gonçalves

[38] Normalização - O processo de organizar bem os dados na base de dados, para reduzir repetições e manter tudo coerente. Segui estes princípios tal como a professora nos ensinou.

[39] 3.ª Forma Normal (3FN) - Um dos níveis da normalização. Resumidamente, uma tabela está na 3FN quando não tem dados repetidos nem colunas que dependam de algo que não seja a chave.

[40] Integridade referencial - A garantia de que as ligações entre tabelas se mantêm sempre coerentes, por exemplo, que uma tarefa não acaba ligada a um utilizador que já não existe.

[41] Chave primária - A coluna que identifica de forma única cada registo de uma tabela, como se fosse o “número de identificação” de cada linha.

[42] Chave estrangeira - A coluna que faz a ponte entre duas tabelas, apontando para a chave primária de outra.

[43] Relação muitos-para-muitos - Quando vários registos de uma tabela podem estar ligados a vários de outra (por exemplo, vários utilizadores em vários grupos). Resolvi estas relações com tabelas intermédias.

[44] ON DELETE CASCADE - Uma regra que, ao apagar um registo, apaga automaticamente os registos que dependiam dele noutras tabelas.

[45] ON DELETE SET NULL - Parecida com a anterior, mas em vez de apagar, deixa o valor a “vazio” (nulo) quando o registo que ele referenciava é eliminado.

[46] JWT (JSON Web Token) - O token de autenticação que o servidor cria depois do login. O cliente guarda-o e envia-o em cada pedido para provar quem é, sem ter de reenviar a palavra-passe.

[47] Hash - O resultado de uma função que transforma dados (como uma palavra-passe) numa sequência irreversível. Usa-se para guardar palavras-passe em segurança, sem ficar com a original.

[48] Salt - Um valor aleatório que se junta à palavra-passe antes do hash, para que duas palavras-passe iguais não gerem o mesmo resultado. É uma camada extra de segurança.

[49] Prepared statements - Uma forma segura de fazer comandos SQL, em que aquilo que o utilizador escreve é tratado sempre como dados e nunca como parte do comando. Foi assim que evitei ataques.

[50] Injeção de SQL - Um tipo de ataque em que alguém tenta “enfiar” código SQL malicioso pelos campos da aplicação, para mexer indevidamente na base de dados. Os prepared statements servem precisamente para o impedir.

[51] N-API e node-gyp - A N-API é o que permite ligar o meu código em C ao Node.js; o node-gyp é a ferramenta que compila esse código C para a aplicação o conseguir usar. Andam de mãos dadas.

[52] Vite - A ferramenta que usei para correr o frontend em React. Ajudou-me bastante, porque atualizava a interface quase instantaneamente, bastava guardar.

[53] Landing Page - A página de entrada de um site ou aplicação, normalmente a primeira que o utilizador vê e que apresenta o produto.

[54] Git - O sistema de controlo de versões que usei para guardar o histórico do código, criar versões paralelas (branches) e poder voltar atrás se algo corresse mal.

[55] Visual Studio Code - O editor de código onde escrevi praticamente todo o projeto. É gratuito e dos mais usados no mundo do desenvolvimento.

Bibliografia

Mozilla Developer Network - Documentação Web (HTTP, DNS). [Mozilla.org](https://developer.mozilla.org) - Acedido em 30/05/2026

Documentação oficial do Node.js. <https://nodejs.org/docs> Acedido em 30/05/2026

Documentação oficial do Express.js. <https://expressjs.com> Acedido em 30/05/2026

Documentação oficial do React. <https://react.dev> Acedido em 30/05/2026

Documentação oficial do MySQL. <https://dev.mysql.com/doc> Acedido em 30/05/2026

Tailwind CSS - *Documentação oficial*. <https://tailwindcss.com/docs> — Acedido em 30/05/2026

Auth0 — *Introduction to JSON Web Tokens*. <https://jwt.io/introduction> — Acedido em 30/05/2026

npm - *bcrypt (biblioteca de hashing de palavras-passe)*. [package/bcrypt](https://npmjs.com/package/bcrypt) - Acedido em 30/05/2026

OWASP Foundation — *SQL Injection Prevention Cheat Sheet*. [SQL Injection Prevention](https://owasp.org/www-project-sql-injection-prevention/) - Acedido em 30/05/2026

Mozilla Developer Network — *HTTPS (HTTP sobre TLS)*. [HTTPS](https://developer.mozilla.org/en-US/docs/Web/Security/HTTPS) — Acedido em 30/05/2026

Material de apoio da disciplina de Redes de Comunicação (Módulos 1-8).

Material de apoio da disciplina de Programação de Sistemas Informáticos (Módulos 1-19).